

Lab 4

- **Launch 3 EC2 instances** in 3 different AZs by choosing subnet (edit)
User data code
- **Create 3 target groups:**
Instances
Name
HTTP, 80
IPV4
Default VPC
HTTP1
Health: HTTP, /
Choose corresponding EC2 instance
Port 80, include
Create
- **Application Load Balancer (1)**
Name
Internet facing
Default VPC
Select the 3 AZs/subnets
Launch1 security group
Forward to target group
Home group, done
- **Add Rules (ALB)**
Conditions, path (/auth, /orders)
Corresponding target group
Priority 1/2, done
- **Health Checks**
HTTP, /, traffic port
Healthy 2, unhealthy 5, timeout 5, interval 30, success 200
- **Launch 3 templates**
Name
Provide guidance
Quick Start, aws, t3.micro
Dont include key pair, subnet, AZ
Launch1 security group
User data code, done
- **3 Auto scaling groups**
Name
Choose template corresponding
Corresponding subnet
Balanced best
Existing LB
Choose from TG
Choose corresponding TG
Desired 1, min 1, max 2
Review, done **[COPY DNS LINK IN ALB]**

Home:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
cat <<EOF > /var/www/html/index.html
<!DOCTYPE html>
<html>
<head><title>My AWS Web App</title></head>
<body>
<h1>Welcome to My AWS Web App</h1>
<p>Homepage served here!</p>
<nav>
<a href="/auth">Authentication Service</a> |
<a href="/order">Order Service</a>
</nav>
</body>
</html>
EOF
```

Auth:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
cat <<EOF > /var/www/html/index.html
<!DOCTYPE html>
<html>
<head><title>Authentication Service</title></head>
<body>
<h1>Authentication Service</h1>
<p>This is the /auth path</p>
</body>
</html>
EOF
```

Orders:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
cat <<EOF > /var/www/html/index.html
<!DOCTYPE html>
<html>
<head><title>Order Service</title></head>
```

```
<body>
<h1>Order Service</h1>
<p>This is the /order path</p>
</body>
</html>
EOF
```

Lab 5

Task1: AWS CloudTrail

- CloudTrail-> Create a trail
- Name and click on create
- Click for details + S3 link
- Create and delete EC2 and S3
- Event History
- A particular event-> JSON log

Task2: Configure CloudWatch for EC2 Instance Monitoring

- Launch EC2 instance

Go to EC2 → Instances → Launch Instances

Name: CloudWatch-Monitor-Instance

AMI: Amazon Linux 2 (Free tier eligible)

Instance type: t2.micro

Key pair: Choose or create one (for SSH)

Network settings: Allow HTTP (port 80) and SSH (port 22)

Storage: Leave default (8 GB), Launch Instance

Go to EC2 → your instance → Monitoring tab

Make sure Detailed Monitoring is enabled (if not, enable it).

- Connect (SSH)

Select the instance → Click Connect

Choose EC2 Instance Connect → Connect

- Commands:

```
sudo yum update -y
```

```
sudo yum install amazon-cloudwatch-agent -y
```

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard
```

[Answer QnA]

```
sudo yum install collectd -y
```

```
sudo mkdir -p /usr/share/collectd
```

```
sudo touch /usr/share/collectd/types.db
```

- **Run:**

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \  
-a fetch-config \  
-m ec2 \  
-c file:/opt/aws/amazon-cloudwatch-agent/bin/config.json \  
-s
```

- **Verify data in CW**

Go to CloudWatch Console → Metrics → All metrics → EC2 → Per-Instance Metrics

You'll see metrics like:

CPUUtilization, DiskReadBytes, DiskWriteBytes, mem_used_percent (if memory metrics are enabled)

- **Create Alarm**

In CloudWatch → Alarms → All alarms → Create alarm

Select metric:

Browse → EC2 → Per-Instance Metrics → Choose CPUUtilization

Choose your instance → Select metric

Conditions:

Threshold type: Static

Whenever CPUUtilization is greater than 70%

For 2 consecutive periods of 1 minute

Actions:

Create a new SNS topic

Topic name: cpu-alert-topic

Enter your email address

Confirm the SNS subscription from your email.

[OR select existing]

Alarm name: HighCPUAlarm

Description: "Triggers when CPU > 70% for 2 mins"

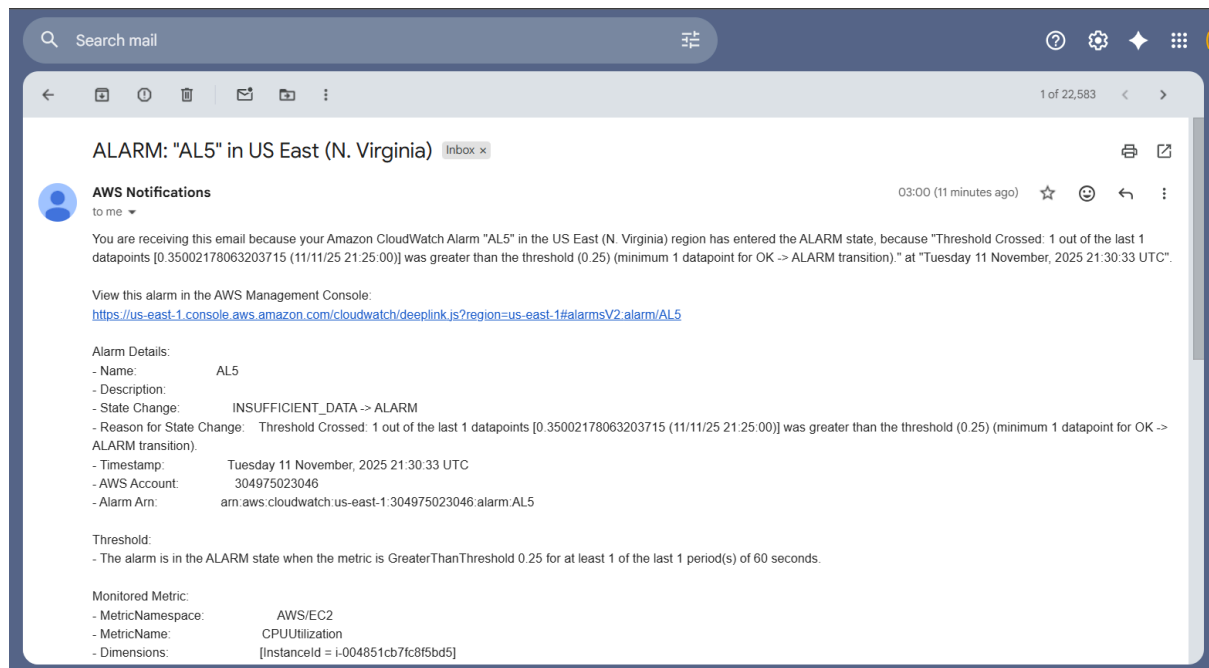
Create alarm

- **Terminal to load:**

```
sudo yum install stress-ng -y
```

```
stress-ng --cpu 4 --cpu-load 90 --timeout 180
```

Wait 3 minutes



MIDTERM QUESTIONS:

Implement EC2 Auto Scaling with a Launch Template. Create a Key Pair and a Security Group allowing HTTP (port 80) and SSH (port 22) access. Create a Launch Template using Amazon Linux 2 with a user-data script to install Apache on startup. Create an Auto Scaling Group attached to the default VPC with a minimum of one instance and a maximum of three instances. Configure scaling policies to scale out when CPU utilization exceeds 50% and scale in when it drops below 20%. Test the scaling behavior by generating CPU load and verify Apache access. Document all steps with screenshots.

1. Create a Key Pair/ Use Ia4

Go to EC2 → Key Pairs → Create key pair.

Name it something like autoscale-key.

Choose Key pair type: RSA

Choose Private key file format: .pem

Click Create key pair.

→ A .pem file downloads automatically. Keep it safe — you'll need it if you SSH in later.

2. Create a Security Group/ Use launch wizard 1

Go to EC2 → Security Groups → Create security group.

Name: asg-sg

Description: Allow HTTP and SSH access

Inbound rules:

Type: SSH, Port: 22, Source: My IP

Type: HTTP, Port: 80, Source: 0.0.0.0/0

Leave outbound rules as default (All traffic allowed).

Click Create security group.

3. Create a Launch Template

Go to EC2 → Launch Templates → Create launch template.

Name: apache-template

Version description: Apache install via user data

AMI: Select Amazon Linux 2 (64-bit x86).

Instance type: t2.micro (Free tier).

Key pair: Select autoscale-key.

Network settings:

Choose your asg-sg Security Group.

Leave subnet unspecified (Auto Scaling will choose).

Make a new security group with ports 80 and 22 allowed

Enable auto assign public IP, and make delete on termination 'Yes'

Advanced details → User data:

Paste this script:

```
#!/bin/bash
```

```
yum update -y
```

```
yum install -y httpd
```

```
systemctl start httpd
```

```
systemctl enable httpd
```

```
echo "<h1>Welcome to Auto Scaling Apache Instance - $(hostname)</h1>" >
```

```
/var/www/html/index.html
```

4. Create an Auto Scaling Group (ASG)

Go to EC2 → Auto Scaling Groups → Create Auto Scaling group.

Name: apache-asg.

Choose the launch template: apache-template.

Click Next.

Network settings:

Choose your default VPC.

Select all subnets in different Availability Zones for better availability.

Load balancing: Skip (None for now).

Group size:

Desired capacity: 1

Minimum capacity: 1

Maximum capacity: 3

Scaling policies:

Choose Target tracking scaling policy

Field	What to choose
-------	----------------

Policy name	cpu-target-scaling
-------------	--------------------

Metric type	Average CPU utilization
-------------	-------------------------

Target value	35
--------------	----

Instance warm-up time	300 seconds (default)
-----------------------	-----------------------

Disable scale-in	Leave unchecked
------------------	-----------------

Launch an EC2 instance, configure MySQL, and create a shopping database. Launch EC2 and install MySQL, then create a database named shopping_db. Create tables customers, products, and orders, insert at least five customer and five product records, and write ten SQL queries for order management. Create foreign key relationships between tables and back up the database.

- Create an EC2 instance **[UBUNTU FOR ANYTHING MYSQL]**
- Connect to SSH

```
sudo apt update -y
sudo apt install mysql-server -y
sudo systemctl start mysql
sudo systemctl enable mysql
sudo mysql
```

SQL CODE:

```
CREATE DATABASE shopping_db;
USE shopping_db;
```

```
CREATE TABLE customers (
  customer_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  email VARCHAR(100),
  phone VARCHAR(20)
);
```

```
CREATE TABLE products (
  product_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  price DECIMAL(10,2),
  stock INT
);
```

```
CREATE TABLE orders (
  order_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT,
  product_id INT,
  quantity INT,
  order_date DATE,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

```
INSERT INTO customers (name, email, phone) VALUES
('Aarav Shah', 'aarav@example.com', '9876543210'),
```

```
('Diya Mehta', 'diya@example.com', '9988776655'),  
( 'Riya Kapoor', 'riya@example.com', '9123456789'),  
( 'Kabir Singh', 'kabir@example.com', '9345678901'),  
( 'Isha Patel', 'isha@example.com', '9001122334');
```

```
INSERT INTO products (name, price, stock) VALUES  
( 'Laptop', 65000.00, 10),  
( 'Headphones', 1500.00, 25),  
( 'Smartwatch', 5000.00, 15),  
( 'Phone Case', 300.00, 40),  
( 'Keyboard', 1200.00, 20);
```

```
INSERT INTO orders (customer_id, product_id, quantity, order_date) VALUES  
(1, 1, 1, '2025-11-11'),  
(2, 2, 2, '2025-11-11'),  
(3, 3, 1, '2025-11-10'),  
(4, 4, 3, '2025-11-09'),  
(5, 5, 1, '2025-11-08');
```

```
-- 1. View all customers  
SELECT * FROM customers;
```

```
-- 2. View all products  
SELECT * FROM products;
```

```
-- 3. View all orders with customer and product details  
SELECT o.order_id, c.name AS customer, p.name AS product, o.quantity, o.order_date  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
JOIN products p ON o.product_id = p.product_id;
```

```
-- 4. Find total number of orders  
SELECT COUNT(*) AS total_orders FROM orders;
```

```
-- 5. Calculate total revenue  
SELECT SUM(p.price * o.quantity) AS total_revenue  
FROM orders o  
JOIN products p ON o.product_id = p.product_id;
```

```
-- 6. Show most popular product  
SELECT p.name, COUNT(*) AS times_ordered  
FROM orders o  
JOIN products p ON o.product_id = p.product_id  
GROUP BY p.name  
ORDER BY times_ordered DESC  
LIMIT 1;
```



```
-- 7. Find customers who ordered more than 1 item
SELECT c.name, COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
HAVING total_orders > 1;
```

```
-- 8. Get total items ordered per product
SELECT p.name, SUM(o.quantity) AS total_sold
FROM products p
JOIN orders o ON p.product_id = o.product_id
GROUP BY p.name;
```

```
-- 9. Update stock after an order (example)
UPDATE products SET stock = stock - 1 WHERE product_id = 2;
```

```
-- 10. Delete an order (example)
DELETE FROM orders WHERE order_id = 5;
```

```
EXIT;
```

```
sudo mysqldump shopping_db > shopping_db_backup.sql
cat shopping_db_backup.sql
```

STATIC WEBSITE CODE:

login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Static Application</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>Login Page</h1>
    <div class="card">
      <input type="text" id="username" placeholder="Username">
      <input type="password" id="password" placeholder="Password">
      <button onclick="login()">Login</button>
    </div>

    <div id="profile-section" class="hidden">
      <h2>Profile</h2>
      <div class="profile-pic">
        
        <input type="file" id="upload" accept="image/*" onchange="previewImage(event)">
      </div>

      <h2>Application Form</h2>
      <form id="appForm" onsubmit="submitForm(event)">
        <input type="text" id="name" placeholder="Full Name" required>
        <input type="email" id="email" placeholder="Email" required>
        <input type="text" id="course" placeholder="Course" required>
        <textarea id="reason" placeholder="Reason for Applying" required></textarea>
        <button type="submit">Submit</button>
      </form>
    </div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

Style.css

```
body {
  font-family: Arial, sans-serif;
  background: #f5f6fa;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.container {
  width: 400px;
  background: #fff;
  padding: 25px;
  border-radius: 12px;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
}

h1, h2 {
  text-align: center;
  color: #2f3640;
}

.card, form {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

input, textarea, button {
  padding: 10px;
  border: 1px solid #dcdde1;
  border-radius: 6px;
  font-size: 14px;
}

button {
  background-color: #273c75;
  color: #fff;
  cursor: pointer;
}

button:hover {
  background-color: #40739e;
}

.profile-pic {
```

```
display: flex;
flex-direction: column;
align-items: center;
margin-bottom: 20px;
}
```

```
.profile-pic img {
border-radius: 50%;
margin-bottom: 10px;
width: 100px;
height: 100px;
}
```

```
.hidden {
display: none;
}
```

script.js

```
function login() {
  const user = document.getElementById('username').value;
  const pass = document.getElementById('password').value;

  if (user === 'admin' && pass === '1234') {
    document.querySelector('.card').classList.add('hidden');
    document.getElementById('profile-section').classList.remove('hidden');
  } else {
    alert('Invalid credentials! Try admin / 1234');
  }
}
```

```
function previewImage(event) {
  const reader = new FileReader();
  reader.onload = function() {
    document.getElementById('preview').src = reader.result;
  };
  reader.readAsDataURL(event.target.files[0]);
}
```

```
function submitForm(event) {
  event.preventDefault();
  const name = document.getElementById('name').value;
  alert('Application submitted successfully!\nThank you, ${name}.');
  document.getElementById('appForm').reset();
}
```